

MP 5149 · UT1 · INTRODUCCIÓ A LES XARXES NEURONALS SUPERVISADES

# Del perceptró al perceptró multicapa

Com funciona una neurona artificial, per què una sola no basta, i com una xarxa aprèn a construir ella mateixa les característiques que necessita.

# Dos mòduls en paral·lel, dues famílies d'algorismes

MP 5134

## Algorismes clàssics

Disseny i avaluació de models

Regressió lineal i logística, k veïns, arbres, boscos aleatoris, SVM, k-means.

Cada algorisme és **una idea concreta** sobre com separar o ajustar les dades. Els feu servir amb scikit-learn i us centreu a **avaluar-los**.

MP 5149

## Xarxes neuronals

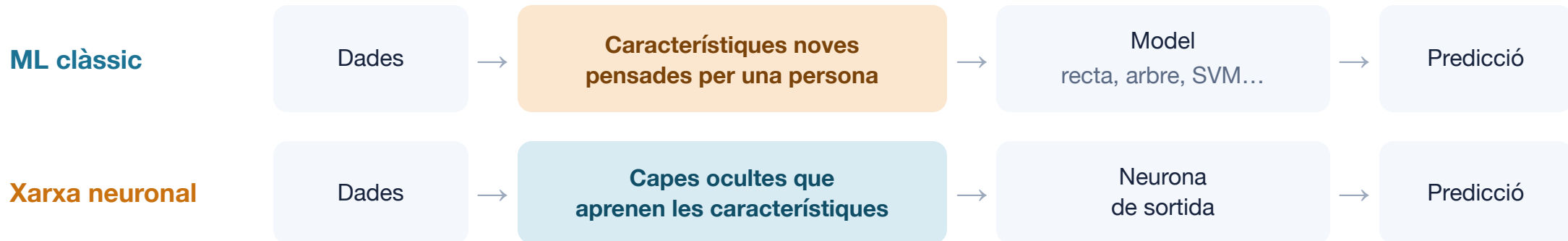
Desenvolupament de components software

Perceptró, perceptró multicapa, mapes autoorganitzatius, xarxes de Hopfield.

Una xarxa és **un joc de construcció** amb peces iguals i simples. Aquí les **programarem per dins**, amb NumPy i classes.

Al 5134 obriu la caixa amb `fit` i `predict` . Al 5149 construïm la caixa.

## Qui fabrica les característiques?



Als algorismes clàssics, les característiques les prepares tu. A les xarxes neuronals, les capes ocultes les aprenen.

Ara sona abstracte. D'aquí a unes quantes diapositives ho veurem passar amb quatre punts.

## DUES FAMÍLIES

# En què es diferencien

|                  | ALGORISMES CLÀSSICS                                                             | XARXES NEURONALS                                                                            |
|------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Què són          | Una idea per algorisme: una recta, preguntes encadenades, el marge més ample    | Moltes neurones iguals i simples, connectades en capes                                      |
| Característiques | Les prepara la persona: polinòmiques, transformacions, codificació              | Les capes ocultes aprenen representacions noves                                             |
| Entrenament      | Un mètode específic per a cada algorisme; molts donen sempre el mateix resultat | Sempre la mateixa recepta: descens del gradient i retropropagació. Depèn de l'atzar inicial |
| On brillen       | Taules de dades estructurades                                                   | Imatges, so, text: dades no estructurades i abundants                                       |
| Cost             | Segons, en qualsevol ordinador                                                  | Sovint necessiten GPU                                                                       |
| Explicació       | Molts es poden llegir: coeficients, arbres                                      | Difícils d'explicar: milers o milions de pesos                                              |

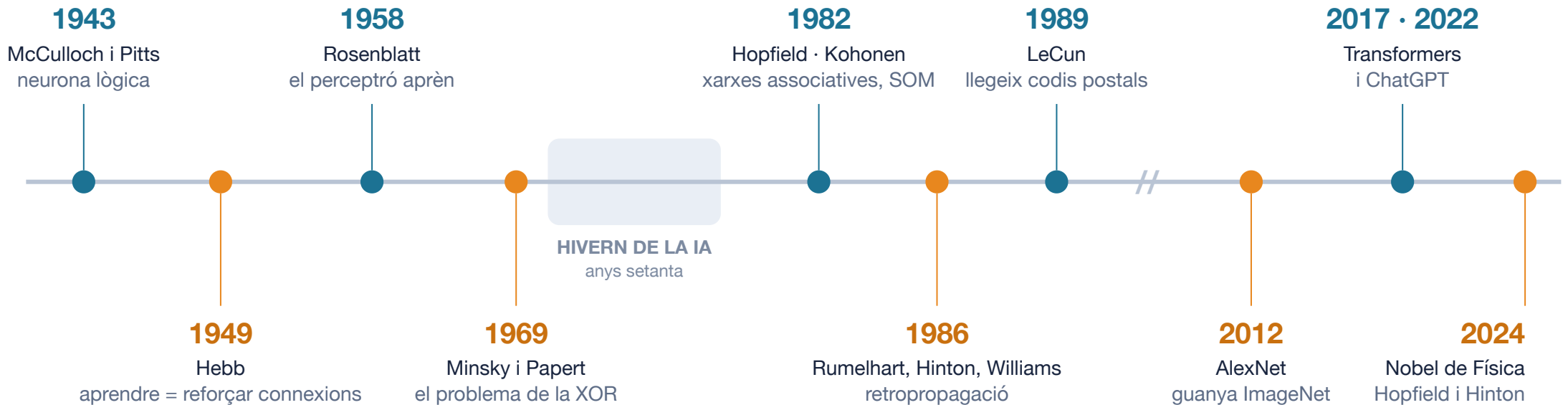
Atenció: **no són sempre millors**. Amb dades tabulars, un bosc aleatori o un *gradient boosting* sovint iguala o supera una xarxa, i és molt més barat.

PART 1

# Vuitanta anys d'eufòria i d'hiverns

La història de les xarxes neuronals és la d'un problema que ningú sabia resoldre, i de la idea que el va desbloquejar.

# Una línia del temps amb alts i baixos



Avui ens quedam entre 1943 i 1986. Hopfield i Kohonen són la UT3 d'aquest mòdul.

# De la neurona biològica a l'artificial

## BIOLOGIA

Senyals que arriben per les **dendrites**

Força de cada **sinapsi**

El **soma** suma els senyals

Es dispara si supera un **llindar**

Aprendre = reforçar sinapsis

## NEURONA ARTIFICIAL

**Entrades**  $x_1, x_2, x_3$

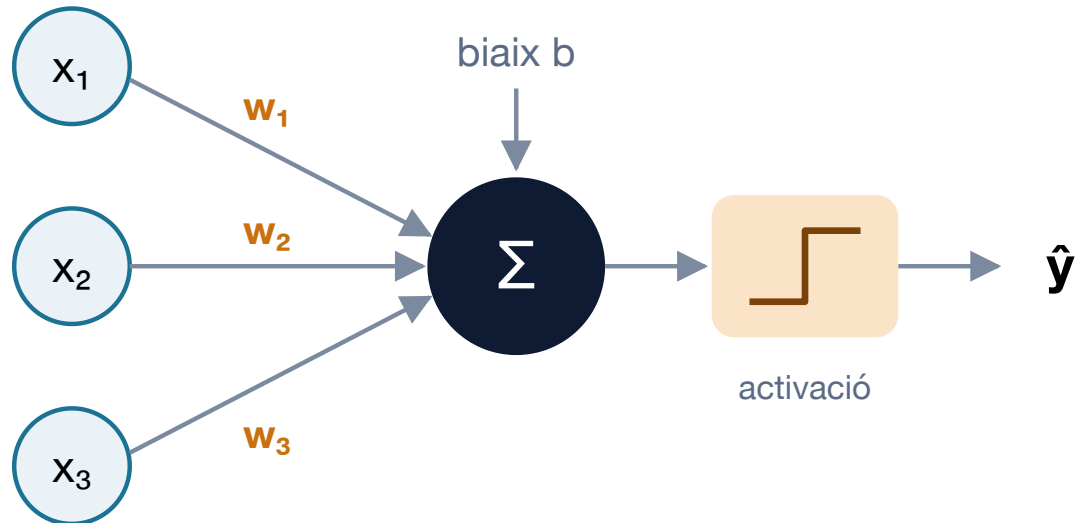
**Pesos**  $w_1, w_2, w_3$

**Suma ponderada**

**Biaix i funció d'activació**

**Entrenar = ajustar pesos**

El cervell té uns 86.000 milions de neurones. Les xarxes artificials se n'**inspiren**, però no el simulen.



# McCulloch i Pitts: una neurona que calcula

Entrades de 0 o 1. La neurona compta quantes n'hi ha d'actives i, si arriba al **llindar**, dispara.

Canviant només el llindar, la mateixa neurona fa portes lògiques diferents. I amb AND, OR i NOT es pot construir **qualsevol càlcul**.

**Però no aprèn res.** El llindar el tries tu. És programació clàssica amb forma de neurona: `DADES + REGLES → RESPOSTES`

| $x_1$ | $x_2$ | AND<br>llindar 2 | OR<br>llindar 1 |
|-------|-------|------------------|-----------------|
| 0     | 0     | 0                | 0               |
| 0     | 1     | 0                | 1               |
| 1     | 0     | 0                | 1               |
| 1     | 1     | 1                | 1               |



# Rosenblatt: el perceptró

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$
$$\hat{y} = 1 \text{ si } z \geq 0, \text{ si no } 0$$

Tres novetats respecte de 1943:

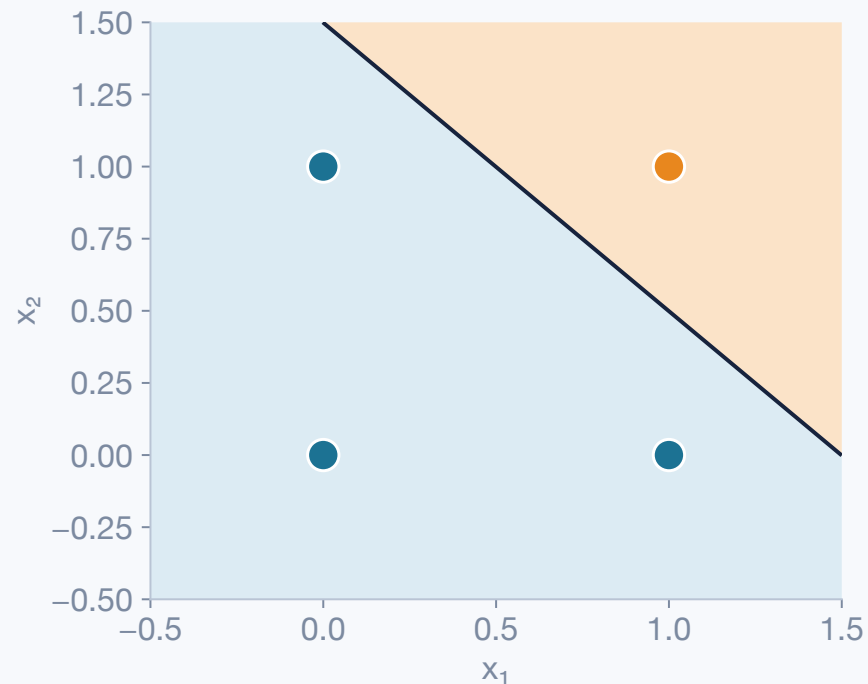
- 1 Les entrades poden ser **qualsevol número**.
- 2 Cada entrada té un **pes**, positiu o negatiu.
- 3 Els pesos **s'aprenen a partir d'exemples**.

## El Mark I Perceptron

No era un programa: era una **màquina**. Una «retina» de 400 cèl·lules fotoelèctriques, i els pesos fets amb **potenciòmetres** que uns motors anaven girant mentre aprenia a reconèixer formes.

La premsa de l'època li va atribuir el futur de caminar, parlar i ser conscient de si mateix. Recordau-ho quan arribem a 1969.

# Una neurona és una recta



L'equació  $\mathbf{w}_1 \cdot \mathbf{x}_1 + \mathbf{w}_2 \cdot \mathbf{x}_2 + \mathbf{b} = 0$  és una recta. Tot el que queda a un costat dóna 1; tot el que queda a l'altre, 0.

Amb  $\mathbf{w} = (1, 1)$  i  $\mathbf{b} = -1,5$  obtenim la porta **AND**: només el punt (1, 1) cau a la zona taronja.

Canvia el biaix a  $-0,5$  i la recta es desplaça: ara és una porta OR. Amb tres entrades seria un pla; amb més, un hiperplà. La idea és la mateixa.

# La regla del perceptró

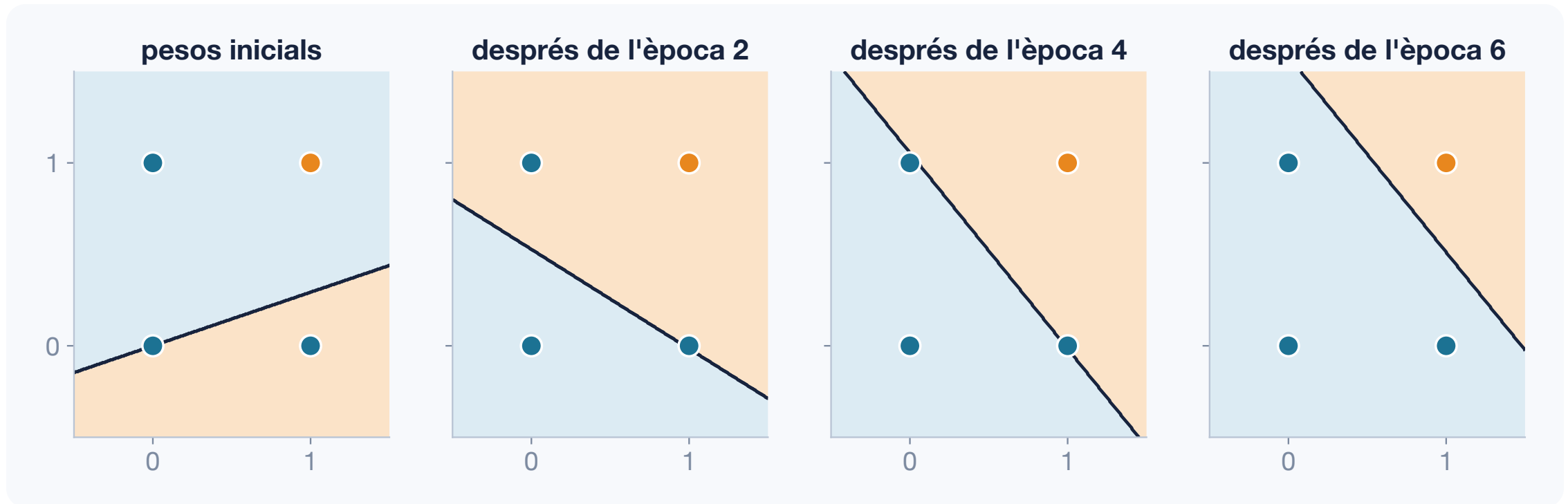
```
# per a cada mostra (x, y), a cada època
 $\hat{y} \leftarrow$  predicció per a x
error  $\leftarrow y - \hat{y}$  # 0, +1 o -1
 $w \leftarrow w + \text{learning\_rate} \cdot \text{error} \cdot x$ 
 $b \leftarrow b + \text{learning\_rate} \cdot \text{error}$ 
```

Una **època** és una passada per totes les mostres. El **coeficient d'aprenentatge** és el primer hiperparàmetre: no s'aprèn, el tries tu.

| SI...                  | ERROR | QUÈ FA                           |
|------------------------|-------|----------------------------------|
| encerta                | 0     | no toca res                      |
| havia de dir 1 i diu 0 | +1    | <b>suma</b> l'entrada als pesos  |
| havia de dir 0 i diu 1 | -1    | <b>resta</b> l'entrada als pesos |

Cada error empeny la recta una mica cap al costat bo per a la mostra que ha fallat.

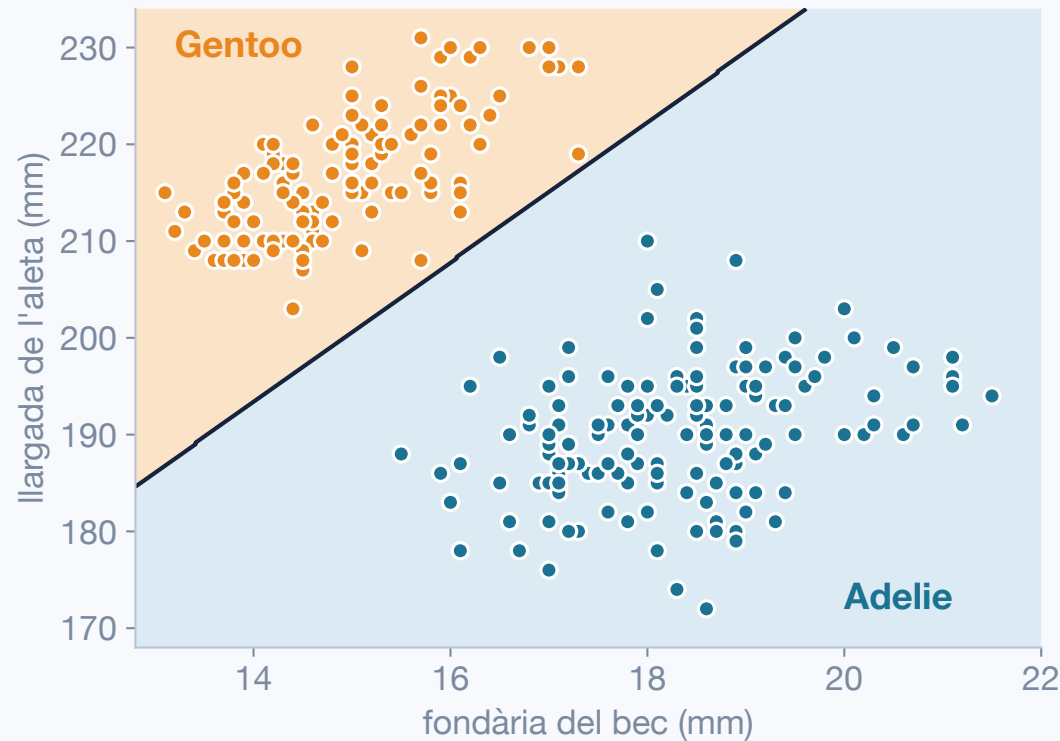
# Veure'l aprendre la porta AND



Errors per època: 2, 3, 2, 3, 2, 1, 0, 0. Els pesos inicials són aleatoris; durant la setena època ja no s'equivoca cap vegada.

**Teorema de convergència:** si existeix una recta que separa les classes, el perceptró la troba en un nombre finit de passos. Fixau-vos en el «sí».

# Adelie o Gentoo, amb una sola neurona



274 pingüins del 5134. Dues mesures. **Encert a test: 100%.**

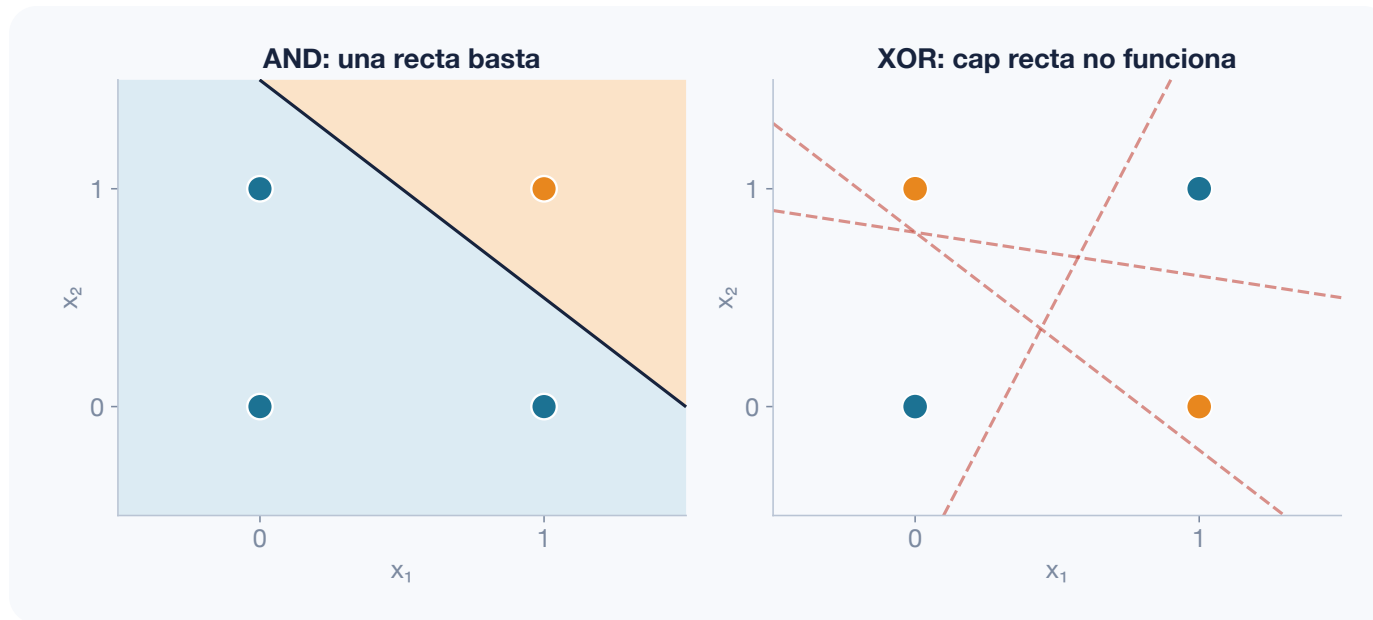
Però mira quant costa aprendre segons l'escala de les entrades. Errors a les primeres èpoques:

|               |    |   |    |   |    |   |   |
|---------------|----|---|----|---|----|---|---|
| mesures en mm | 93 | · | 94 | · | 25 | · | 0 |
|---------------|----|---|----|---|----|---|---|

|                   |   |   |   |   |   |   |   |
|-------------------|---|---|---|---|---|---|---|
| mesures escalades | 3 | · | 0 | · | 0 | · | 0 |
|-------------------|---|---|---|---|---|---|---|

L'aleta fa ~200 mm i el bec ~17: les correccions d'un pes són deu vegades les de l'altre. **A les xarxes neuronals, escalar no és opcional.**

# El mur de la XOR



XOR val 1 quan **una i només una** entrada és 1.

Els punts taronges són a una diagonal i els blaus a l'altra: **no hi ha cap recta** que els separi. El perceptró hi pot estar èpoques i èpoques.

Minsky i Papert ho van analitzar al llibre *Perceptrons*. La retallada de finançament que va seguir es coneix com el **primer hivern de la IA**.

# Com ho resoldria un algorisme clàssic?

| $x_1$ | $x_2$ | $x_1 \cdot x_2$ | XOR |
|-------|-------|-----------------|-----|
| 0     | 0     | 0               | 0   |
| 0     | 1     | 0               | 1   |
| 1     | 0     | 0               | 1   |
| 1     | 1     | 1               | 0   |

Al 5134 la resposta és **fabricar una característica nova**: afegim la columna  $x_1 \cdot x_2$  i el mateix perceptró ja ho resol.

Però algú ha hagut de **pensar** que el producte era la columna que calia. Amb quatre punts és fàcil. Amb una foto d'un milió de píxels, quina columna inventes per saber si hi ha un gat?

I si la xarxa aprengué sola quines característiques necessita?

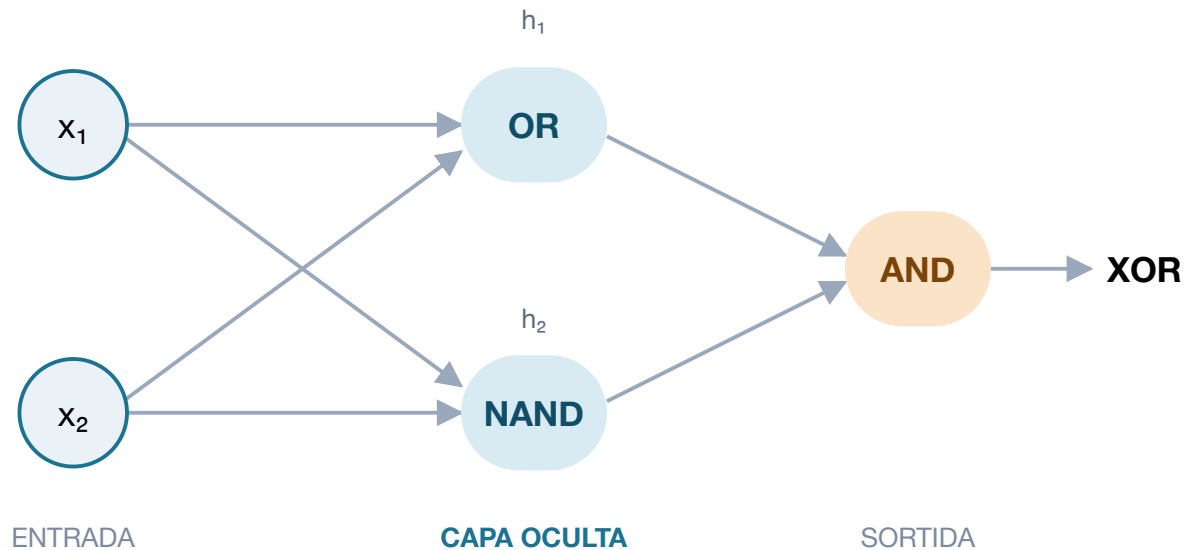
## PART 2

# La solució: afegir capes

Posar neurones entre l'entrada i la sortida. Primer ho farem a mà, per entendre per què funciona. Després, que ho aprengui sola.



## La XOR amb una capa oculta

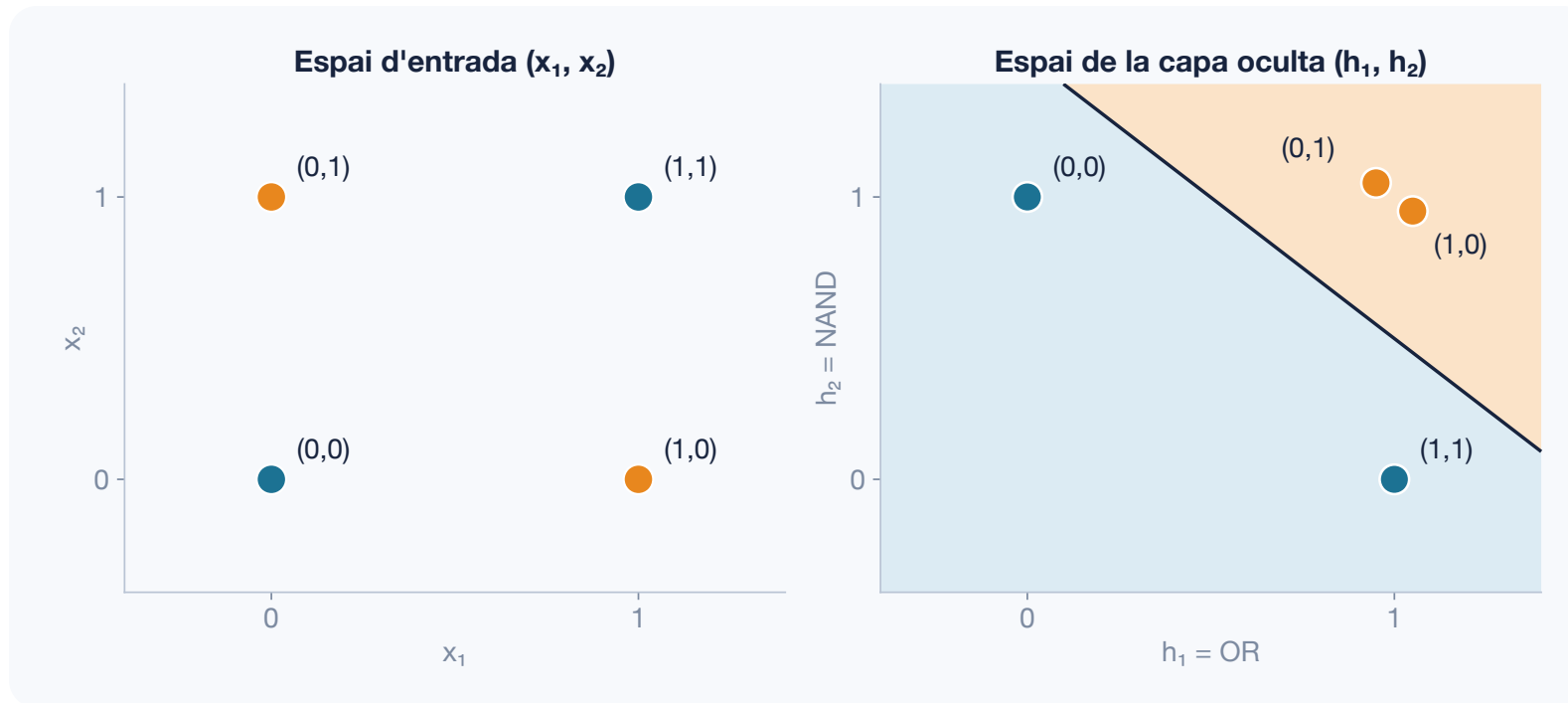


$$\text{XOR} = (x_1 \text{ OR } x_2) \text{ AND NOT } (x_1 \text{ AND } x_2)$$

Cada caixa és un perceptró normal, amb pesos triats a mà. Cap no fa res que un perceptró no sàpiga fer.

Les neurones del mig no es veuen ni a l'entrada ni a la sortida: formen una **capa oculta**. Això ja és un **perceptró multicapa** (MLP).

# La capa oculta transforma el problema



A l'esquerra, el problema impossible. A la dreta, **els mateixos quatre punts** vistos per la capa oculta: els taronges cauen junts i una recta ja els separa.

La capa oculta ha **creat característiques noves**,  $h_1$  i  $h_2$ , igual que fèiem a mà amb  $x_1 \cdot x_2$ . La diferència: en una xarxa entrenada **les tria l'entrenament**.

# Per què va costar disset anys?

Saber que les capes funcionen no diu **com trobar-ne els pesos**. Hi havia dos obstacles.

## 1. A qui tiram la culpa?

La regla del perceptró compara la sortida amb la resposta correcta. Però una neurona oculta **no té resposta correcta**: ningú no li ha dit que havia de fer OR.

Si la xarxa s'equivoca, quina neurona oculta en té la culpa, i quanta?

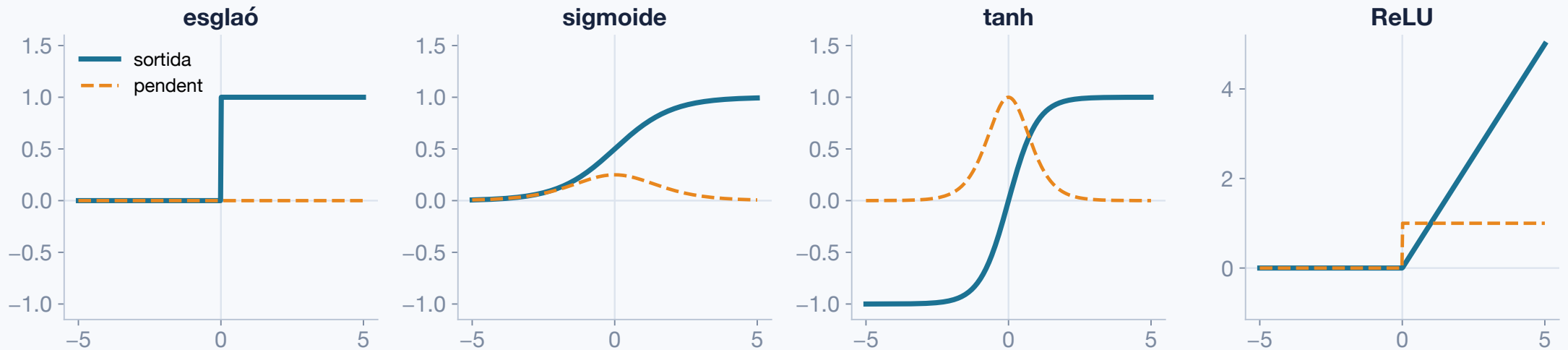
## 2. L'esglaó és pla

Si mous un pes una mica, la sortida continua sent 0 o 1: **no canvia gens**.

Sense cap canvi, no hi ha pista de cap a on moure el pes per millorar.

La solució té tres peces: **activacions suaus, descens del gradient i retropropagació**.

# Funcions d'activació suaus



**Esglaó.** Pendent zero: no dóna cap pista. No serveix per entrenar capes.

**Sigmoide.** De 0 a 1, es llegeix com una probabilitat. A la sortida.

**tanh.** De -1 a 1, centrada. La farem servir a la capa oculta.

**ReLU.** L'opció per defecte de les xarxes profundes actuals.

Per al 5134: **una neurona amb sigmoide és exactament una regressió logística.** El que la converteix en xarxa és apilar-ne moltes.

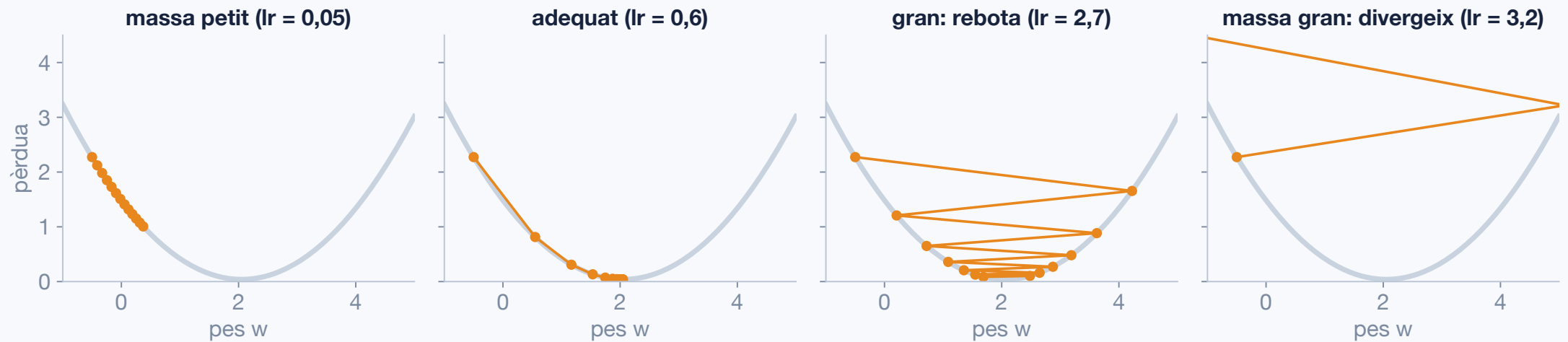
# Pèrdua i descens del gradient

La **funció de pèrdua** diu com de malament ho fa la xarxa.  
Al 5134 va veure que dibuixa una **vall**.

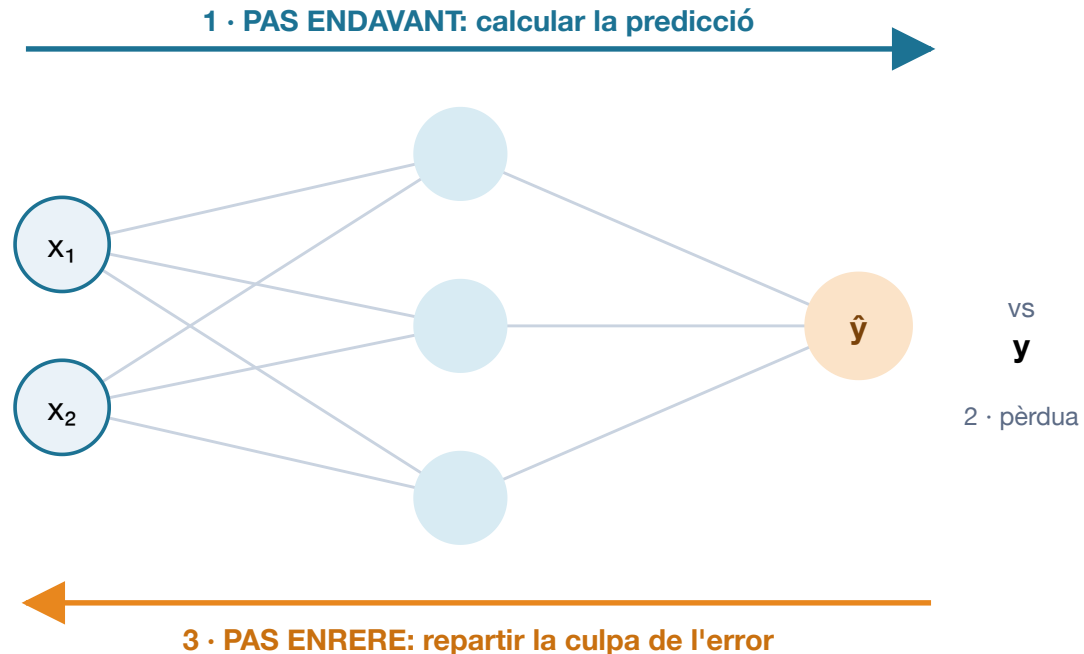
Enmig de la boira, per baixar mires cap a on fa pendent i fas una passa cap avall.

$$w \leftarrow w - \text{learning\_rate} \cdot \text{pendent}$$

El conjunt de pendents de tots els pesos és el **gradient**.



# Retropropagació: repartir la culpa cap enrere



La sortida sap exactament quant s'ha equivocat:  $\hat{y} - y$ . És la mateixa idea que l'error del perceptró.

Cada neurona oculta en rep una part proporcional a:

- el **pes** que la uneix amb la sortida (si el seu vot pesava molt, té molta culpa)
- el **pendent** de la seva activació (si estava saturada, poca)

4 · Amb la culpa de cada neurona, fem una passa de descens del gradient a cada pes.

## El bucle d'entrenament

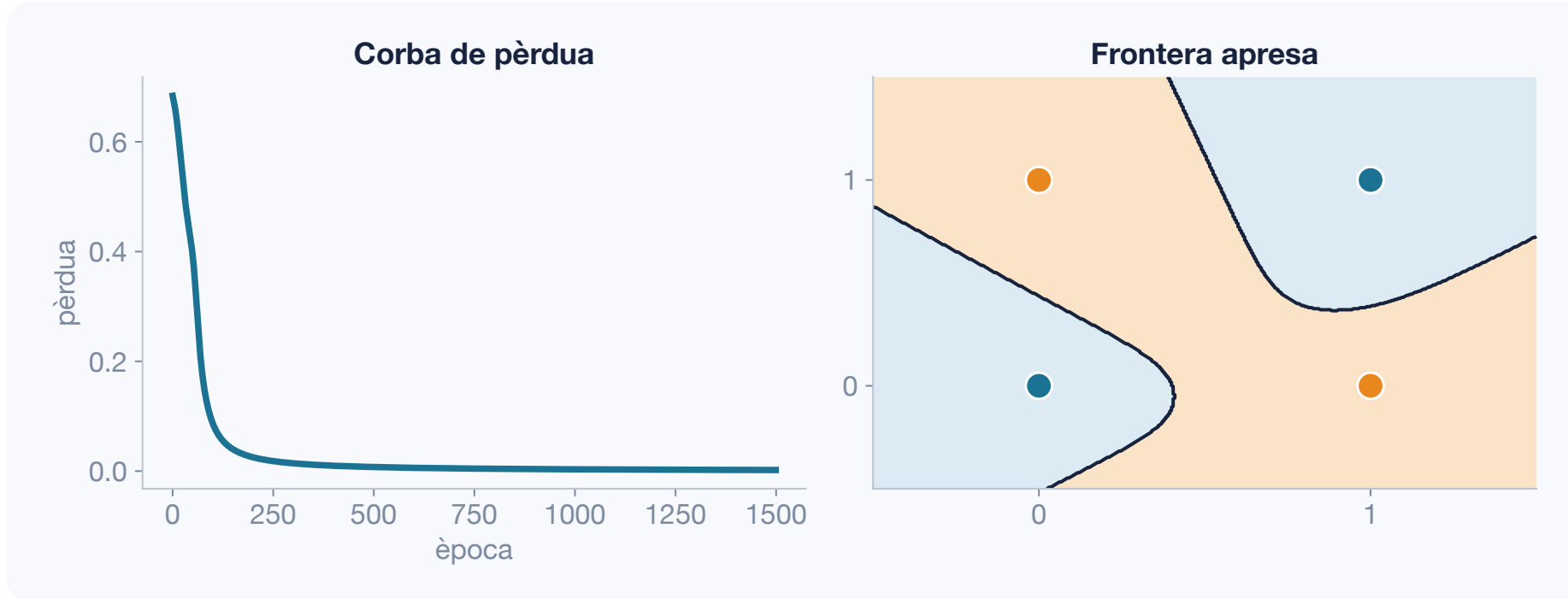
```
repetir durant n_epochs:
    # 1. pas endavant
    a1 ← tanh(X · W1 + b1)
    a2 ← sigmoide(a1 · W2 + b2)
    # 2. pèrdua
    loss ← entropia_creuada(y, a2)
    # 3. pas enrere
    delta2 ← a2 - y
    delta1 ← (delta2 · W2) × (1 - a12)
    # 4. actualització
    W2 ← W2 - lr · a1 · delta2
    W1 ← W1 - lr · X · delta1
```

Entrenar una xarxa és **repetir aquests quatre passos milers de vegades**.

Les xarxes que reconeixen imatges o generen text fan aquest mateix bucle, amb més capes, més neurones i més dades.

**Abans de fiar-nos-en, una prova.** Movem cada pes una quantitat minúscula i mirem quant canvia la pèrdua. Si coincideix amb el que calcula la retropropagació, el codi és correcte.

## L'MLP aprèn la XOR tot sol

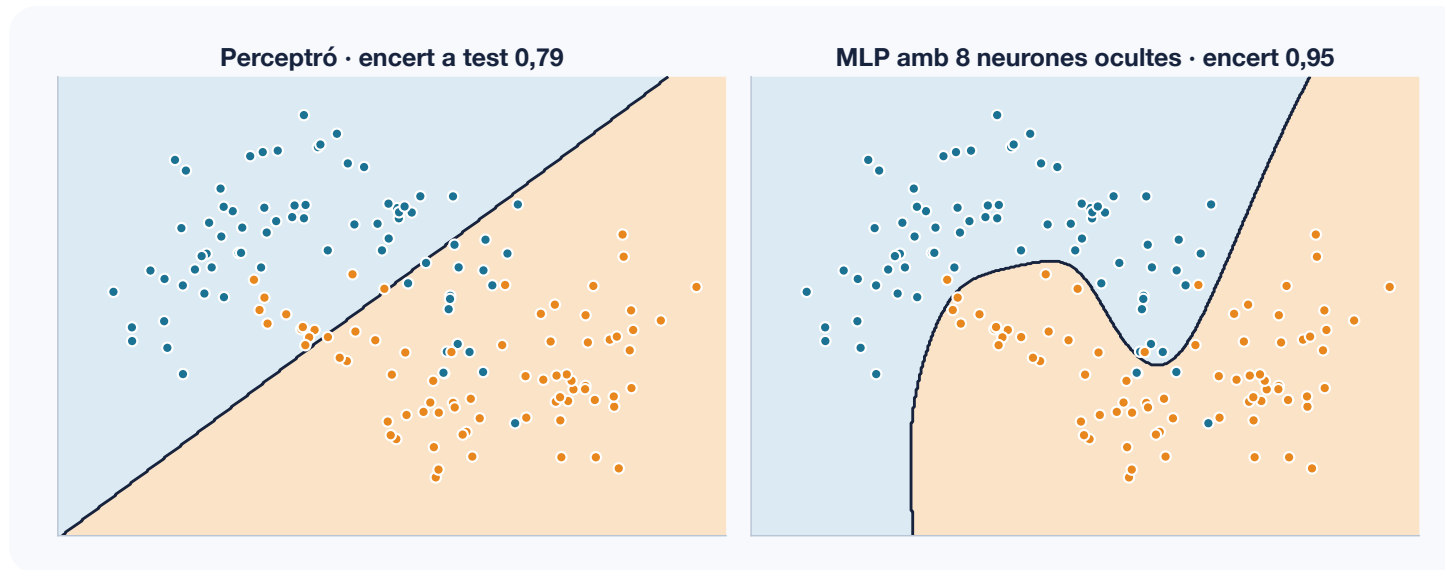


No li hem dit que calien portes OR i NAND ni quins pesos fan falta. Només les quatre files i les respostes.

La corba de pèrdua és el primer gràfic que s'ha de mirar sempre: aquí cau en picat les primeres èpoques i després s'aplana a prop de zero. I la frontera **ja no és una recta**.



# Les dues llunes

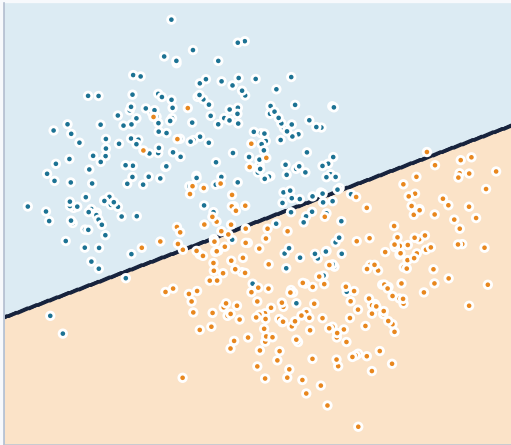


| MODEL                  | ENCERT A TEST |
|------------------------|---------------|
| Referència (Dummy)     | 0,50          |
| Regressió logística    | 0,86          |
| El nostre perceptró    | 0,79          |
| <b>El nostre MLP</b>   | <b>0,95</b>   |
| MLPClassifier (8)      | 0,94          |
| MLPClassifier (16, 16) | 0,95          |

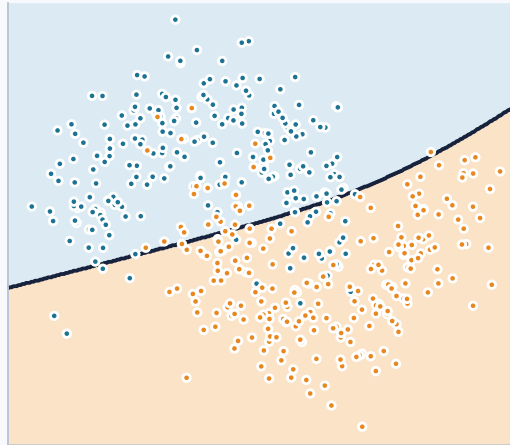
Logística i perceptró són una sola neurona: una recta. No és que siguin dolents, és que el problema no és lineal.

# Neurones, coeficient d'aprenentatge i atzar

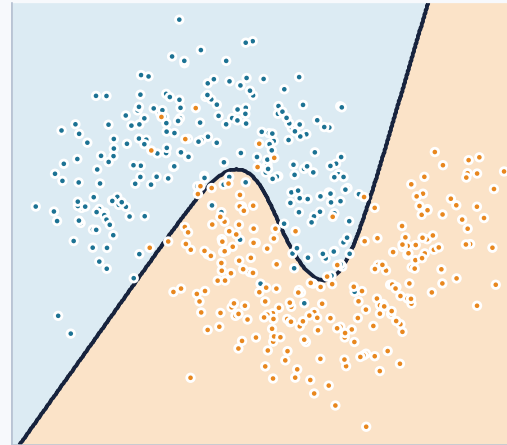
1 neurona · test 0,86



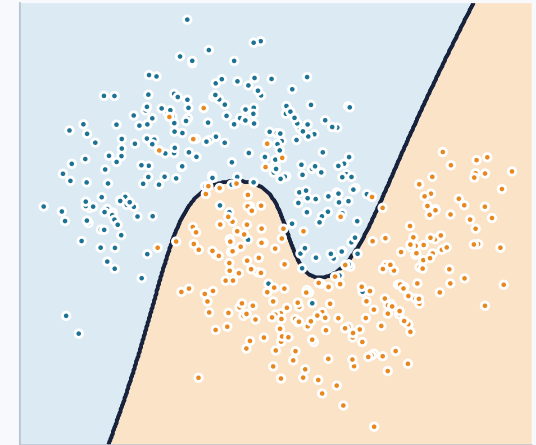
2 neurones · test 0,87



4 neurones · test 0,94



32 neurones · test 0,94



**Neurones ocultes.** Amb poques, la frontera no es pot corbar. Amb moltes, pot acabar memoritzant el soroll: el **sobreajust**.

**Coeficient d'aprenentatge.** Probablement el més important. Massa petit no avança; massa gran rebota o divergeix. Es diagnostica mirant la corba de pèrdua.

**La llavor aleatòria.** Amb 2 neurones ocultes, la XOR s'encalla o no segons els pesos inicials: un **mínim local**. Una regressió lineal sempre dona el mateix.

# El mateix component, amb la mateixa interfície

## Programat per nosaltres

```
model = MLP(n_hidden=8,  
            learning_rate=0.5,  
            n_epochs=3000)  
model.fit(X_train, y_train)  
model.score(X_test, y_test)  
model.loss_curve_
```

## scikit-learn

```
model = MLPClassifier(hidden_layer_sizes=(8,),  
                      activation="tanh",  
                      learning_rate_init=0.01)  
model.fit(X_train, y_train)  
model.score(X_test, y_test)  
model.loss_curve_
```

Com que tots dos tenen `fit`, `predict` i `score`, **es poden intercanviar sense tocar la resta del codi**. Això és dissenyar un component.

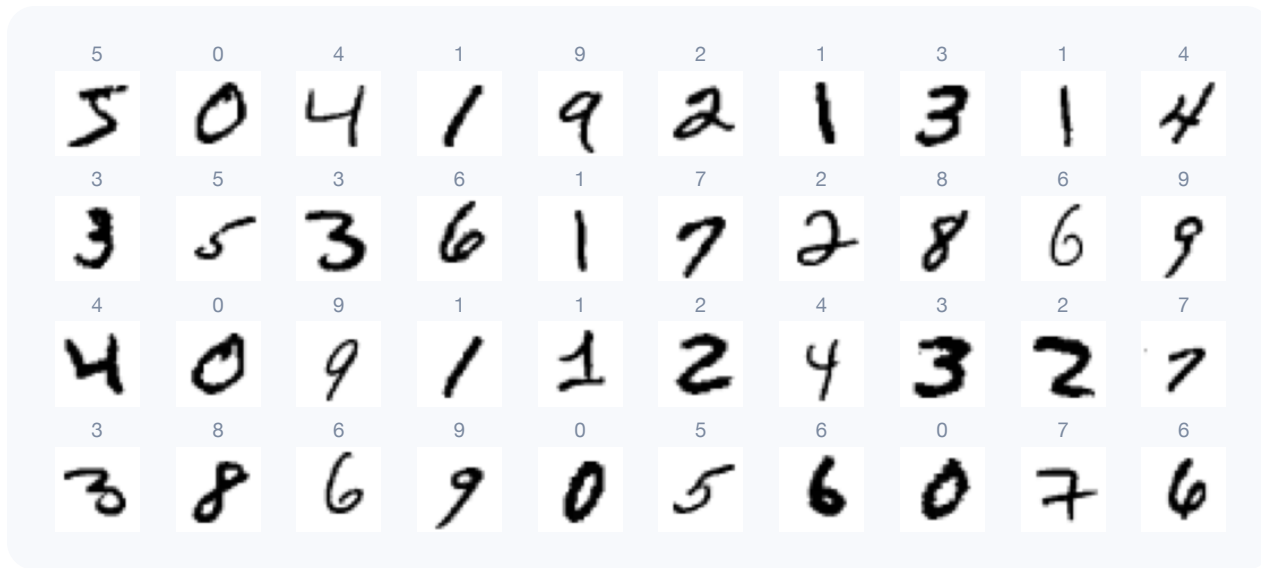
scikit-learn fa servir l'optimitzador **Adam** i lots de mostres, però la recepta de fons és la mateixa: pas endavant, pèrdua, pas enrere, actualització.

# Dades de veritat: 70.000 dígits escrits a mà

Portem el nostre MLP de les llunes a MNIST. Canviar d'escala obliga a introduir tres idees noves, i ens deixa veure què aprenen de debò les neurones.

### 3 · MNIST

Una imatge és una fila de 784 números



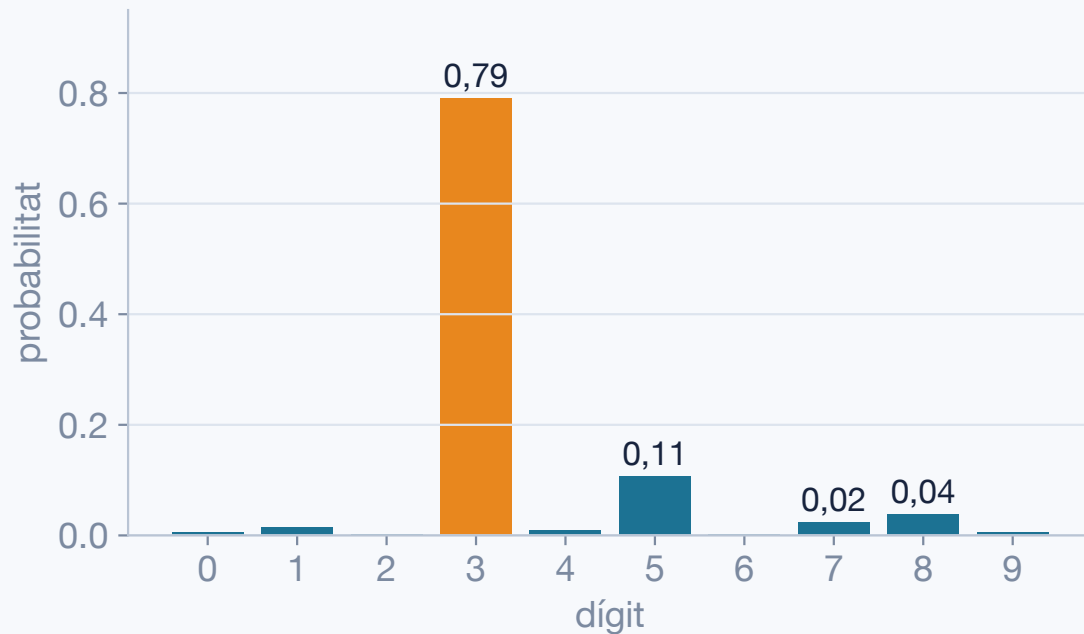
**28 × 28 píxels** de 0 (paper) a 255 (tinta),  
posats en fila: **784 columnes**.

|               |        |
|---------------|--------|
| Entrenament   | 50.000 |
| Validació     | 10.000 |
| Test segellat | 10.000 |

LeCun i col·laboradors, 1998. Durant anys, *la* prova de referència de la visió per computador. A scikit-learn:

```
fetch_openml("mnist_784") .
```

## De 2 classes a 10: softmax



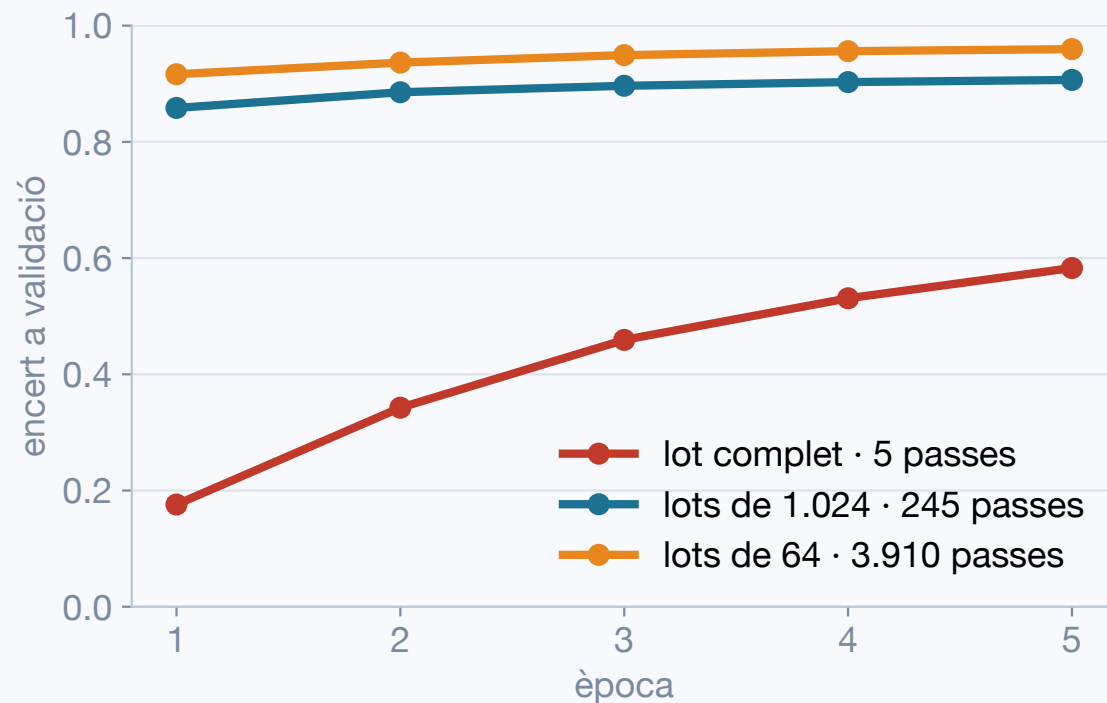
**Deu neurones de sortida**, una per dígit. La **softmax** converteix les deu puntuacions en deu probabilitats que sumen 1.

La resposta es codifica en **one-hot**: deu posicions, totes a 0 menys la correcta.

**El pas enrere no canvia.** Amb softmax i entropia creuada, la culpa de la sortida continua sent  $a_2 - y$ .

La classe nova **hereta** la retropropagació del NB 1.1 sense tocar-ne cap línia.

## Mini-lots: moltes passes per època



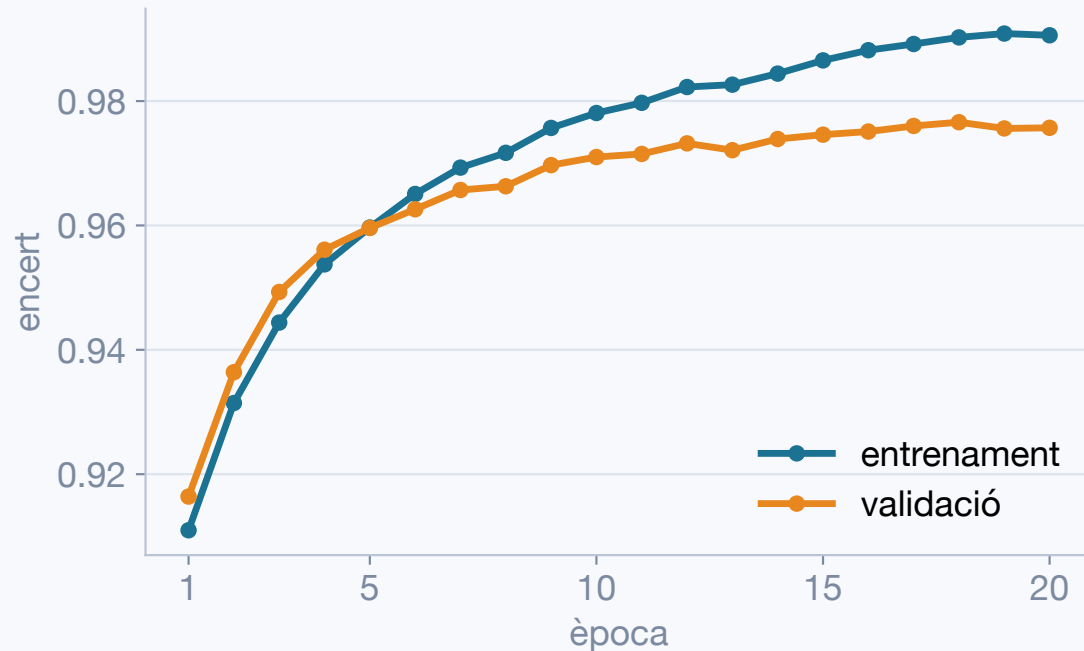
Barrejar, partir en lots de 64 i fer **una passa per lot**.

Mateixes dades, mateixes 5 èpoques i un temps semblant. La diferència és quantes vegades es corregeixen els pesos.

|                 |                                      |
|-----------------|--------------------------------------|
| <b>Iteració</b> | un lot processat i una actualització |
|-----------------|--------------------------------------|

|              |                                 |
|--------------|---------------------------------|
| <b>Època</b> | totes les dades: 782 iteracions |
|--------------|---------------------------------|

## Validació durant l'entrenament



Xarxa **784-128-10**: més de **100.000 pesos**. La de la XOR en tenia 13.

Programada per nosaltres amb NumPy: **97,6% a validació** en 20 èpoques.

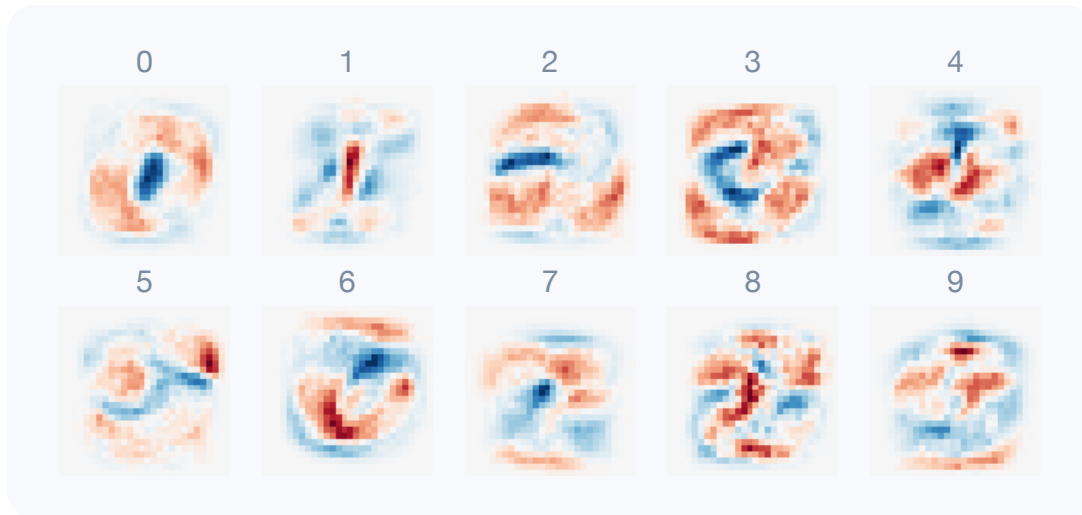
Al principi les dues corbes pugen juntes. Cap al final l'entrenament continua millorant i la validació s'atura: comença el **sobreajust**.

Test segellat, obert al final: **97,45%**. L'estimació de validació era fiable.



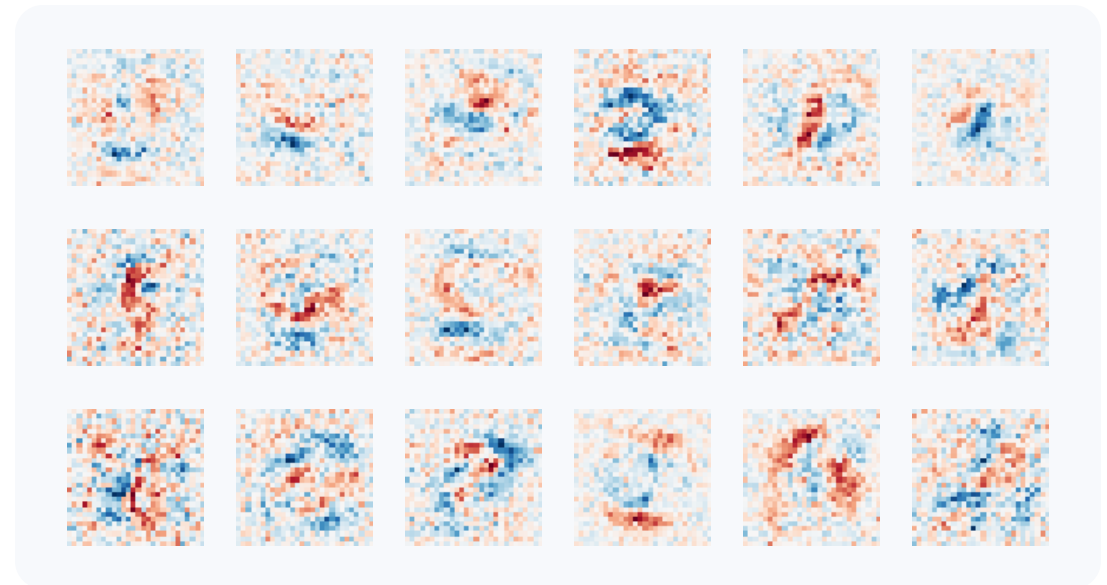
## Plantilles contra traços

### REGRESSIÓ LOGÍSTICA



Un model lineal aprèn **una plantilla per dígit** i hi compara cada imatge.

### NEURONES OCULTES DE L'MLP



Cap no és un dígit: són **traços** en una zona. La sortida combina quins traços hi ha. **Ningú no li ha dit que busqués traços.**

## Les característiques apreses serveixen

El mateix model lineal, una regressió logística, amb tres entrades diferents:

784 píxels originals

92,8%

Una frontera lineal sobre píxels no dona per més.

128 característiques **apreses** per la capa oculta

97,7%

Sis vegades menys columnes i molt més encert.

128 característiques d'una xarxa **sense entrenar**

91,0%

El control: passar per neurones no fa màgia.

És la figura de la XOR a gran escala: la capa oculta transforma les dades en una representació on un model lineal ja funciona.

# Les xarxes guanyen, però per poc

| MODEL                      | ENCERT A VALIDACIÓ |
|----------------------------|--------------------|
| Referència (Dummy)         | 10,6%              |
| Regressió logística        | 92,8%              |
| Bosc aleatori (100 arbres) | 97,2%              |
| <b>El nostre MLP (128)</b> | <b>97,6%</b>       |
| <b>MLPClassifier (128)</b> | <b>97,8%</b>       |

Un bosc aleatori, sense capes ocultes, queda a menys d'un punt.

## EL LÍMIT: DESPLAÇAR 3 PÍXELS



|                     |               |
|---------------------|---------------|
| Regressió logística | 92,8% → 42,2% |
| Bosc aleatori       | 97,2% → 54,2% |
| El nostre MLP       | 97,6% → 53,8% |

Per a l'MLP la imatge és una taula: **no sap quins píxels són veïns**. Les xarxes **convolucional**s reutilitzen el mateix detector a totes les posicions, i llavors la distància es fa enorme.

## RESUM

# Les sis idees que us heu d'endur

- 1 Una **neurona** fa una suma ponderada, hi suma un biaix i aplica una activació. Separa l'espai amb una recta.
- 2 El **perceptró** aprèn corregint els pesos quan s'equivoca, però només resol problemes **linealment separables**.
- 3 Les **capes ocultes** transformen les dades en una representació on el problema sí que és separable.
- 4 Per entrenar capes calen **activacions suaus**, una **pèrdua** i el **descens del gradient**.
- 5 La **retropropagació** reparteix la culpa de l'error cap enrere per saber com moure cada pes.
- 6 Amb dades reals calen **softmax**, **mini-lots** i **validació**. La capa oculta aprèn característiques, i l'**arquitectura decideix quines** pot aprendre.

# Cinc preguntes sense una única resposta

- 1 Si una regressió logística és una neurona, per què diem que les xarxes són una **família diferent**?
- 2 El 1958 es va prometre massa i va venir un hivern. Hi veieu **paral·lelismes amb avui**?
- 3 Una xarxa encerta el 98% però no s'explica; un arbre encerta el 94% i es llegeix. Quin triaríeu per **concedir un crèdit**? I per **trobar pingüins en fotos aèries**?
- 4 La retropropagació és de 1986 i les xarxes no dominen fins al 2012. **Què faltava**?
- 5 A MNIST, un bosc aleatori gairebé iguala l'MLP. Per què, doncs, les xarxes **han guanyat en imatges**?